CLOUD-06: GCP Integration

> **Series:** CLOUD | **Notebook:** 6 of 8 | **Created:** March 2026 | **Last Updated:** 03/12/2026

Overview

This notebook covers Dynatrace integration with Google Cloud Platform (GCP). You will learn how to configure GCP monitoring, authenticate via service accounts, explore supported GCP services, query GCP entities and metrics, and monitor GKE and Cloud Run workloads.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `logs.read`
GCP Project	With service account configured for Dynatrace
Connection	Clouds app or Helm-based GKE integration (recommended for SaaS) or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. GCP Integration Architecture

Dynatrace integrates with GCP through the Cloud Monitoring API (formerly Stackdriver). For SaaS deployments, the recommended approach uses a **push-based Pub/Sub integration** deployed via Helm on GKE — no ActiveGate required.

Connection Methods

Method	Mechanism	Best For
Helm on GKE (push-based)	Metrics and logs forwarded via Pub/Sub to Dynatrace API	SaaS deployments (recommended)
Classic ActiveGate polling	Queries Cloud Monitoring API	Managed deployments, non-GKE environments
Cloud Functions + Pub/Sub	Forwards logs via Pub/Sub trigger	Log-only ingestion

Data Flow (Modern — Push-Based)

```
GCP Resources → Cloud Monitoring → Pub/Sub → Dynatrace Integration (on GKE) → Dynatrace SaaS
GCP Resources → Cloud Logging    → Pub/Sub → Dynatrace Integration (on GKE) → Dynatrace SaaS
```

Helm Deployment Options

Option	Cluster Type	Notes
New GKE Autopilot cluster	Autopilot	Recommended — Dynatrace manages the cluster
Existing GKE cluster	Standard or Autopilot	Deploy into an existing cluster

The Helm deployment creates the necessary Pub/Sub subscriptions, IAM roles, and forwarding components automatically.

GCP-Specific Considerations

- **Project-based organization** — GCP organizes resources by project, unlike AWS regions or Azure subscriptions
- **Labels vs tags** — GCP uses "labels" (equivalent to AWS tags / Azure tags)
- **Cloud Monitoring quotas** — API call limits apply per project
- **Pub/Sub costs** — Message volume determines Pub/Sub costs; filter at source to reduce volume

2. Service Account Authentication

GCP integration requires a service account with appropriate IAM roles.

Setup Steps

1. **Create a service account** in the GCP project
2. **Assign IAM roles** (see table below)
3. **Generate a JSON key** for the service account

4. **Upload the key** to Dynatrace Settings > Cloud and virtualization > GCP

Required IAM Roles

Role	Purpose
`roles/monitoring.viewer`	Read Cloud Monitoring metrics
`roles/compute.viewer`	Read Compute Engine resources
`roles/container.viewer`	Read GKE clusters and workloads
`roles/cloudasset.viewer`	Read asset inventory for entity discovery

Multi-Project Monitoring

To monitor multiple GCP projects with a single integration:

Approach	How It Works
Service account per project	Each project has its own SA; configure multiple integrations in Dynatrace
Cross-project service account	Grant SA roles at the organization/folder level; single integration
Monitoring scope	Configure a Cloud Monitoring metrics scope to aggregate across projects

> **Best Practice:** Use a dedicated GCP project for monitoring resources (service accounts, Pub/Sub topics) to keep billing and IAM clean.

3. Supported GCP Services

Category	Services
Compute	Compute Engine (GCE), Cloud Functions, App Engine
Containers	GKE (Google Kubernetes Engine), Cloud Run
Database	Cloud SQL, Cloud Spanner, Bigtable, Firestore
Storage	Cloud Storage, Persistent Disk
Data	BigQuery, Pub/Sub, Dataflow, Dataproc
Networking	Cloud Load Balancing, Cloud CDN, Cloud DNS
AI/ML	Vertex AI, Cloud TPU

Service Metric Availability

Service	Key Metrics	Typical Delay
Compute Engine	CPU, memory, disk, network	3-5 minutes
GKE	Container CPU/memory, pod status	1-3 minutes
Cloud Run	Request count, latency, instance count	1-3 minutes
Cloud Functions	Execution count, duration, memory usage	1-3 minutes
Cloud SQL	CPU, connections, storage, replication lag	3-5 minutes
BigQuery	Slot utilization, query count, bytes processed	5-10 minutes

4. Querying GCP Entities

List GCP-Hosted Entities

GCP compute instances that run OneAgent appear as regular host entities. Cloud-specific entities appear under their dedicated entity types.

```
// List all hosts (includes GCE instances with OneAgent)
fetch dt.entity.host
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20
```

List Kubernetes Clusters (Including GKE)

```
// List all Kubernetes clusters (GKE clusters appear here)
fetch dt.entity.kubernetes_cluster
| fieldsKeep id, entity.name, tags
| sort entity.name asc
```

GCP Service Entity Count

```
// Count cloud application entities (includes GKE workloads, Cloud Run)
fetch dt.entity.cloud_application
| summarize resource_count = count()
| fieldsAdd resource_type = "Cloud Applications (GKE/Cloud Run)"
```

5. GKE Monitoring

Google Kubernetes Engine (GKE) is monitored with the DynaKube Operator, similar to EKS and AKS.

GKE-Specific Considerations

Feature	Monitoring Impact
Autopilot mode	No node-level access; use ApplicationMonitoring mode
Standard mode	Full node access; use CloudNativeFullStack mode
Workload Identity	Recommended for GKE pod IAM (replaces service account keys)
GKE Dataplane V2	eBPF-based networking; compatible with Dynatrace

GKE Container Metrics

```
// Top 10 containers by CPU usage in the last hour
timeseries containerCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.container.name}
| fieldsAdd avgCpu = arrayAvg(containerCpu)
| sort avgCpu desc
| limit 10
```

GKE Node Utilization

```
// Node CPU utilization across GKE nodes in the last hour
timeseries nodeCpu = avg(dt.kubernetes.node.cpu_usage), from:-1h, by:
{dt.entity.kubernetes_node}
| fieldsAdd avgNodeCpu = arrayAvg(nodeCpu)
| sort avgNodeCpu desc
| limit 10
```

GKE Pod Events

```
// Kubernetes warning events in the last 6 hours
fetch events, from:-6h
| filter event.kind == "K8S_EVENT" and event.type == "Warning"
| summarize event_count = count(), by:{event.reason}
| sort event_count desc
| limit 10
```

6. Cloud Run Analysis

Cloud Run is GCP's serverless container platform. It shares some monitoring patterns with Lambda but runs containers instead of functions.

Cloud Run Monitoring Points

Metric	Description	Alert Threshold
Request count	Total HTTP requests	Baseline deviation
Request latency	Response time (p50, p95, p99)	SLO-dependent
Instance count	Active container instances	Max instance limit
CPU utilization	Per-instance CPU usage	> 80% sustained
Memory utilization	Per-instance memory	> 80% (risk of OOM)
Startup latency	Cold start time for new instances	Application-dependent

Cloud Run vs Lambda

Aspect	Cloud Run	AWS Lambda
Unit	Container	Function
Runtime	Any Docker image	Managed runtimes + custom
Max duration	60 minutes	15 minutes
Concurrency	Multiple requests per instance	One per execution (default)
Monitoring	OneAgent in container	Lambda Layer
Cold start	Container pull + startup	Runtime initialization

```
// Cloud Run service spans in the last hour
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(cloud.platform) and cloud.platform == "gcp_cloud_run"
| summarize avg_duration_ms = avg(duration) / 1000000.0, request_count =
count(), by:{service.name}
| sort request_count desc
| limit 10
```

7. GCP-Specific Entity Mapping

GCP uses a project-based hierarchy that maps to Dynatrace as follows:

GCP Construct	Dynatrace Mapping	Notes
Organization	Account-level (manual)	Not auto-discovered
Folder	Tag or Management Zone	Organizational grouping
Project	Tag (`gcp.project`)	Primary scoping unit
Region / Zone	Tag or entity property	Geographic placement
Labels	Dynatrace tags	Key-value metadata

Best Practices for GCP Mapping

Practice	Description
Tag by project	Ensure GCP project ID is propagated as a Dynatrace tag
Use label conventions	Standard labels: `env` , `team` , `service` , `cost-center`
Segment by project	Create Dynatrace segments per GCP project for access control
Monitor billing exports	Forward BigQuery billing data to Dynatrace for cost correlation

8. Summary and Next Steps

Key Takeaways

- GCP integration authenticates via **service account JSON keys** with Monitoring Viewer and Compute Viewer roles
- **GKE Autopilot** requires ApplicationMonitoring mode; **GKE Standard** supports CloudNativeFullStack
- **Cloud Run** is monitored as a serverless container platform with OneAgent injected into the container image
- GCP **projects and labels** should map to Dynatrace tags and segments for consistent governance

Next Steps

- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding patterns (applicable to GCP via Pub/Sub)
- **CLOUD-08: Multi-Cloud Patterns** — Unified monitoring across GCP and other providers

---